EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 14

tegrating Object, Dynamic and Behavioural Models into Object-Oriented Desig

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 14 of 16

Contents

1. Week 14 Introduction

Students have travelled a long way: from *what problem does the organisation have?* through *what objects exist, how do they relate, behave and interact?* to Week 13's *how should the software be organised into layers and components?* Week 14 now reaches a critical question: **how do we combine all these models into one coherent software design?**

The official UML specification treats structural modelling, behavioural modelling, use cases, deployments and related modelling concepts as parts of the same overall modelling language — so students should not think of UML diagrams as independent drawings. **Each diagram provides a different view of the same system.**

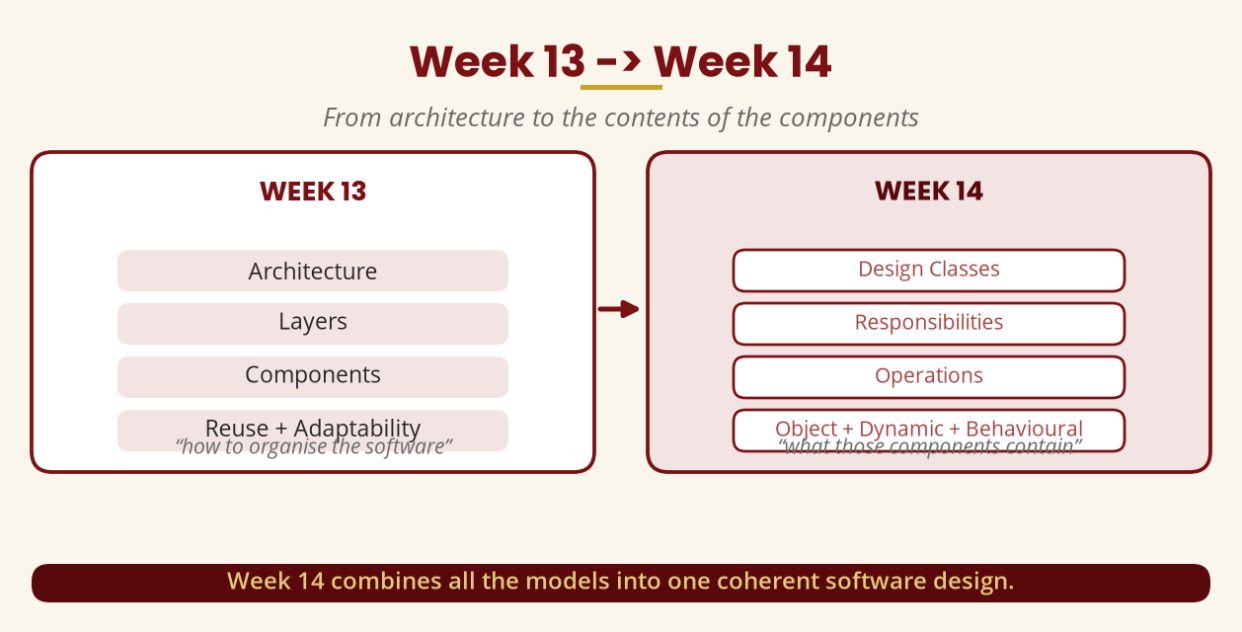

Figure 1 — Week 13 -> Week 14

2. Learning Outcomes

By the end of this week, a student should be able to:

- Define object-oriented design and explain the difference between analysis and design.
- Explain why analysis models must be transformed into design models.
- Explain design classes and identify their responsibilities.
- Assign operations to appropriate classes and explain attributes vs operations.
- Integrate the object model with the dynamic model and with behavioural models.
- Trace a requirement from a use case to classes and operations.
- Use sequence, state and activity diagrams to refine class responsibilities.
- Identify design inconsistencies and refine an analysis model into a design model.

PART A — WHAT IS OBJECT-ORIENTED DESIGN?

3. Analysis vs Design

Object-oriented design (OOD) is the process of transforming the understanding of a system developed during analysis into a detailed design that can guide implementation. **Analysis tells us what the system needs and what exists; design decides how the software will actually be organised to provide it.** If someone tells an architect *"I need a house with three bedrooms, a kitchen, two bathrooms and a garage"*, that is requirements and analysis; the architect then decides where rooms, doors, electricity and pipes go and what materials are needed — that is design.

Given *"a student must register for a course"*, analysis identifies Student, Course and Registration; design introduces RegistrationController, RegistrationService, RegistrationRepository and RegistrationValidator. The design is more implementation-oriented.

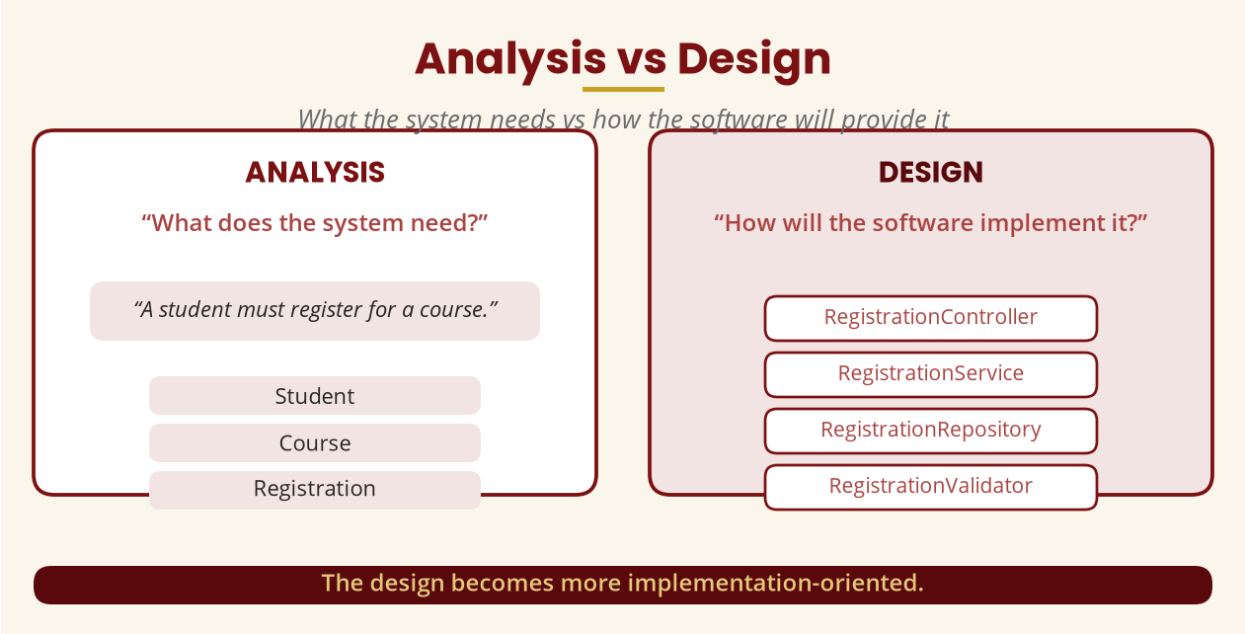

Figure 2 — Analysis vs design

4. Analysis Model -› Design Model

Requirements -> Use cases -> Analysis model -> Design decisions -> Design model -> Implementation. The important point: **we should not randomly invent design classes** — they should be justified by the requirements and analysis.

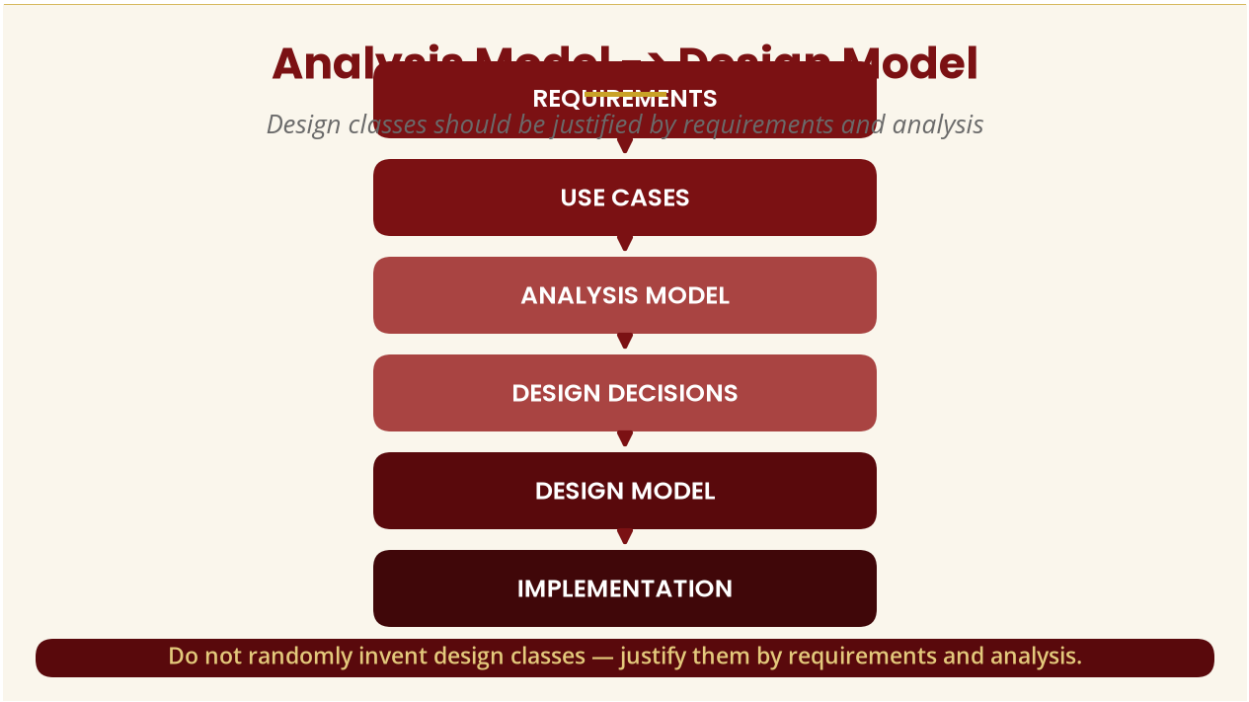

Figure 3 — Analysis -> design transformation

PART B — THE THREE MAJOR MODELS

5. Object, Dynamic and Behavioural Models

The **object model** describes the structural/static view — what things exist and how they relate (classes, objects, attributes, operations, associations, generalisation, aggregation/composition). The **dynamic model** focuses on how objects behave and change over time — Course: Available -> Full -> Closed, with events such as “student registers” or “course reaches capacity.” The **behavioural model** focuses on what the system does from the users' perspective — use cases, use-case diagrams, sequence diagrams, communication diagrams and activity diagrams.

Why do we need all three? A class diagram alone tells us the structure but not what happens when a student registers; a sequence diagram shows the interaction; a state model shows state-dependent behaviour. **Different UML models answer different questions.**

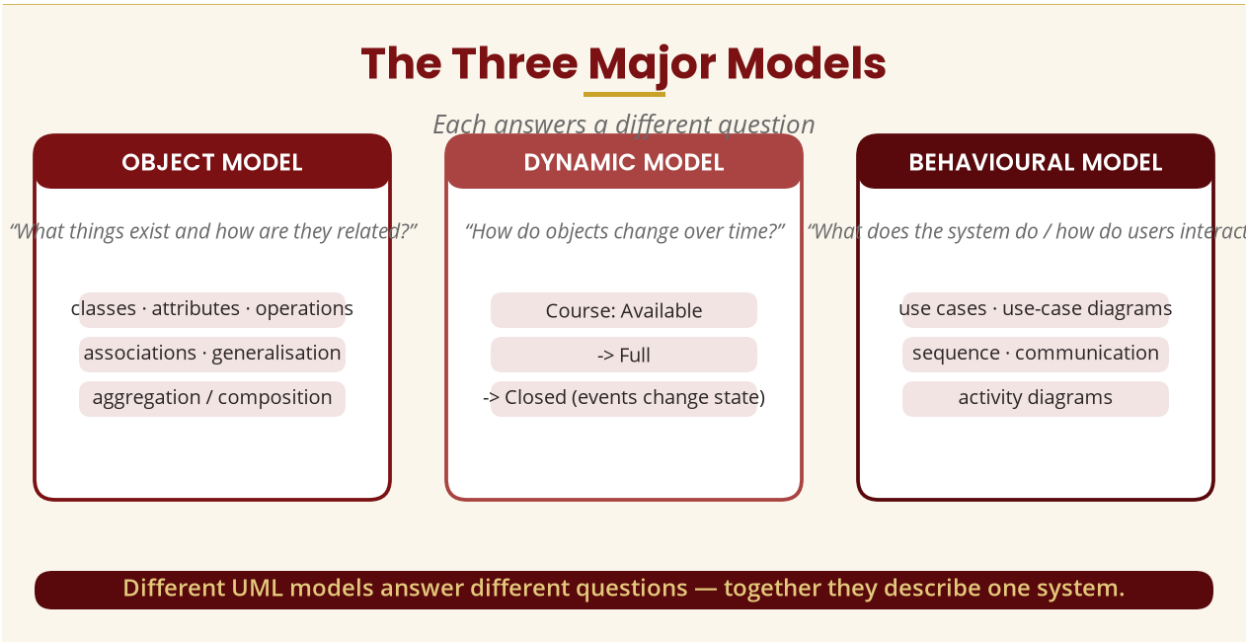

Figure 4 — The three major models

PART C — ONE SYSTEM, MULTIPLE VIEWS

6. Six Views of One System

Modelling a University Course Registration System: the **use-case diagram** tells who uses the system and what they want; the **class diagram** tells what structural elements exist; the **sequence diagram** tells how objects interact in a scenario; the **activity diagram** tells how a workflow progresses; the **state machine** tells how an object changes state; the **component diagram** tells how functionality is modularised. These are not six unrelated systems — they are six views of one system.

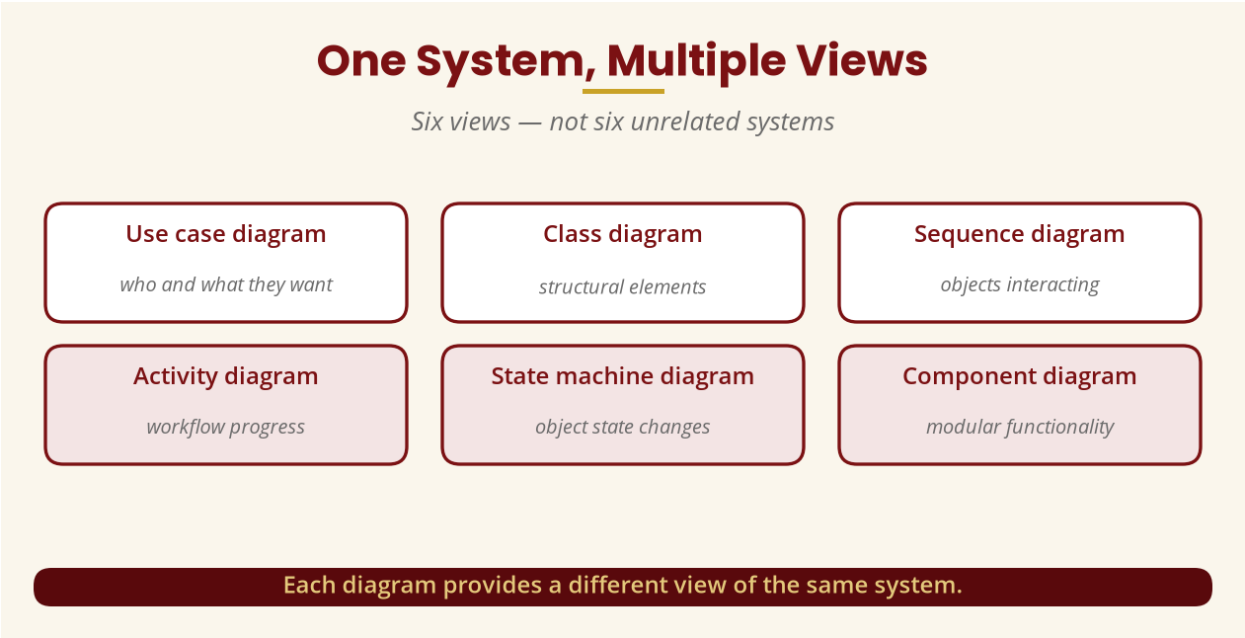

Figure 5 — One system, multiple views

PART D — FROM USE CASE TO DESIGN

7. The Basic Use-Case Flow and Candidate Classes

For **Register for Course** (actor Student): (1) student logs in; (2) selects a course; (3) system checks the course exists; (4) checks prerequisites; (5) checks capacity; (6) creates registration; (7) confirms. From this we identify the domain classes Student, Course and Registration, plus the design classes RegistrationController, RegistrationService, CourseRepository and RegistrationRepository. The designer asks: **which objects should perform these responsibilities?**

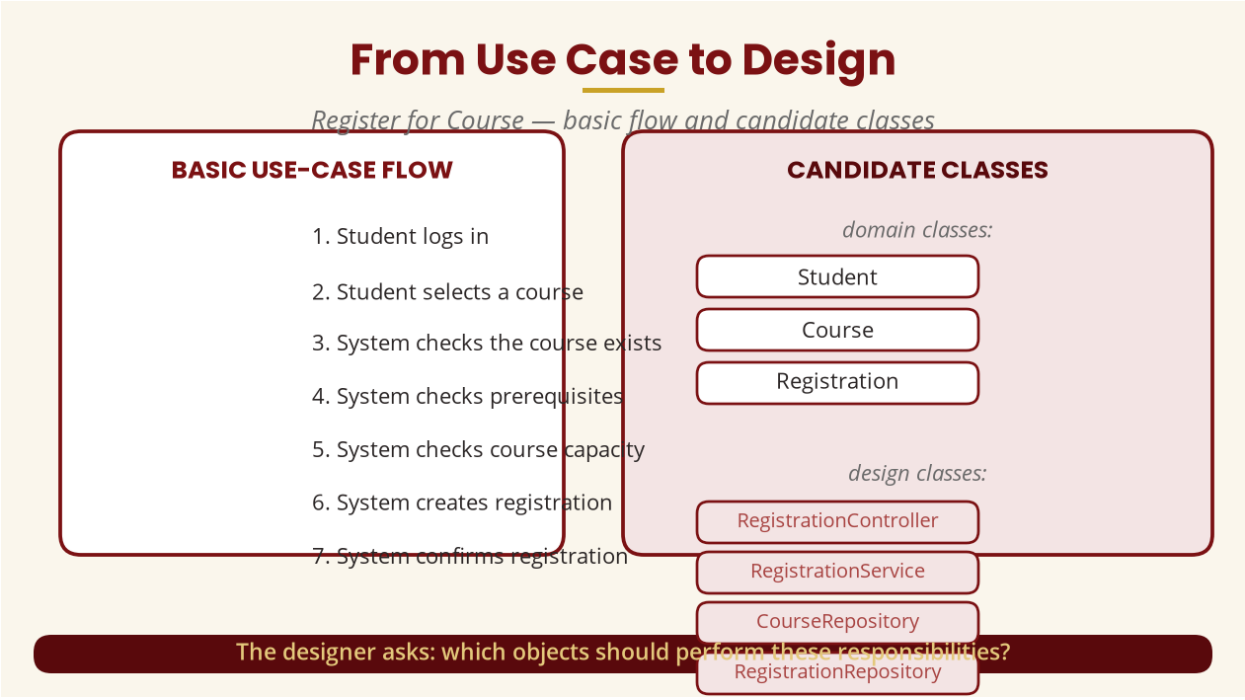

Figure 6 — From use case to design

PART E — RESPONSIBILITY ASSIGNMENT

8. Knowing and Doing

A **responsibility** is something a class is responsible for **knowing** (Student knows studentId, name, programme; Course knows courseCode, courseName, capacity) or **doing** (Course checks whether space is available). A Student class that also does registerAnyCourse(), calculateCourseCapacity(), sendEmail(), generatePaymentReceipt() and connectToDatabase() has poor cohesion — a better assignment routes through RegistrationService -> Course -> RegistrationRepository so each class has a focused purpose.

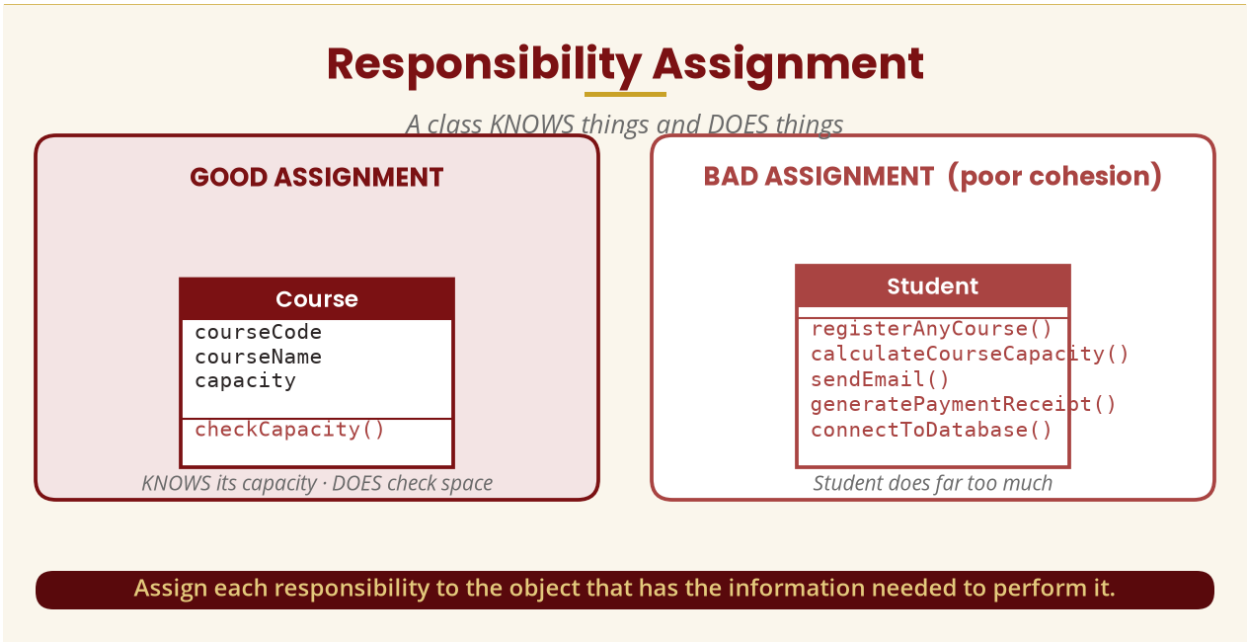

Figure 7 — Responsibility assignment

PART F & G — DESIGN CLASSES AND VISIBILITY

9. Analysis Class -> Design Class

A **design class** is defined with sufficient detail to support implementation: precise attributes, data types, operations, parameter types, return types, visibility, relationships, interfaces and dependencies. The analysis class `Student (name, email, register())` becomes `Student { -studentId: String; -name: String; -email: String; -programme: Programme; +register(course: Course): Registration; +drop(course: Course): void }`.

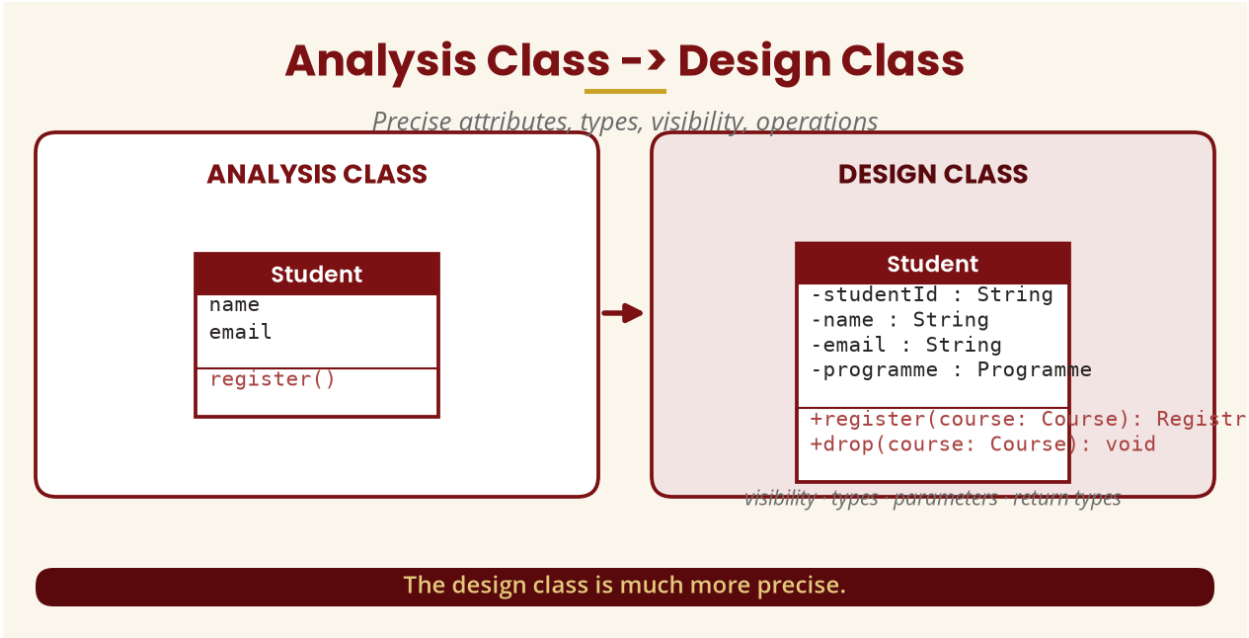

Figure 8 — Analysis class -> design class

10. Visibility

Common UML visibility: + public, - private, # protected, ~ package visibility — e.g. `-studentId : String` and `+getName() : String`.

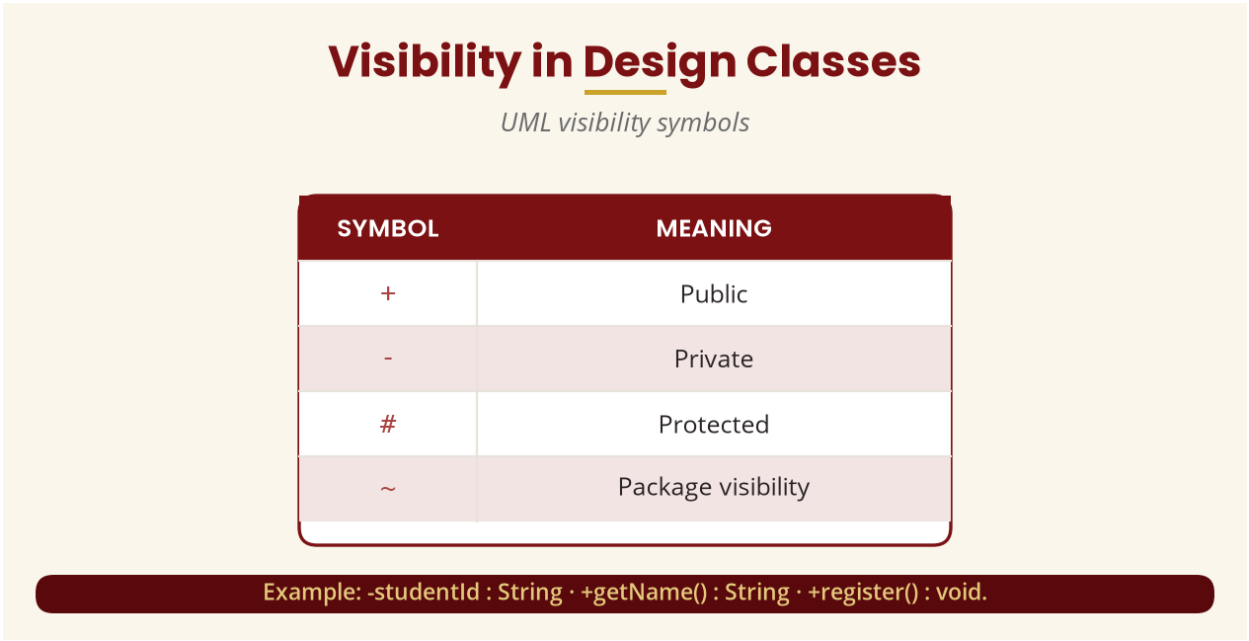

Figure 9 — Visibility symbols

PART H — ATTRIBUTES AND OPERATIONS

Attributes describe what an object knows (studentId, name, email, dateOfBirth); **operations** describe what it can do (register(), dropCourse(), viewCourses()). A BankAccount has attributes `-accountNumber : String` and `-balance : Decimal` and operations `+deposit(amount): void`, `+withdraw(amount): boolean`, `+getBalance(): Decimal` — attributes store information, operations perform actions.

PART I — SEQUENCE DIAGRAM -> CLASS DESIGN

11. From Messages to Operations

A sequence diagram is not only for presentation — it helps discover **which objects need which operations**. If the sequence shows Course -> checkCapacity(), the Course class should probably have `checkCapacity()`; if RegistrationService -> registerStudent(), then `registerStudent()` is required. **Messages in interaction diagrams can help refine operations in the class model.**

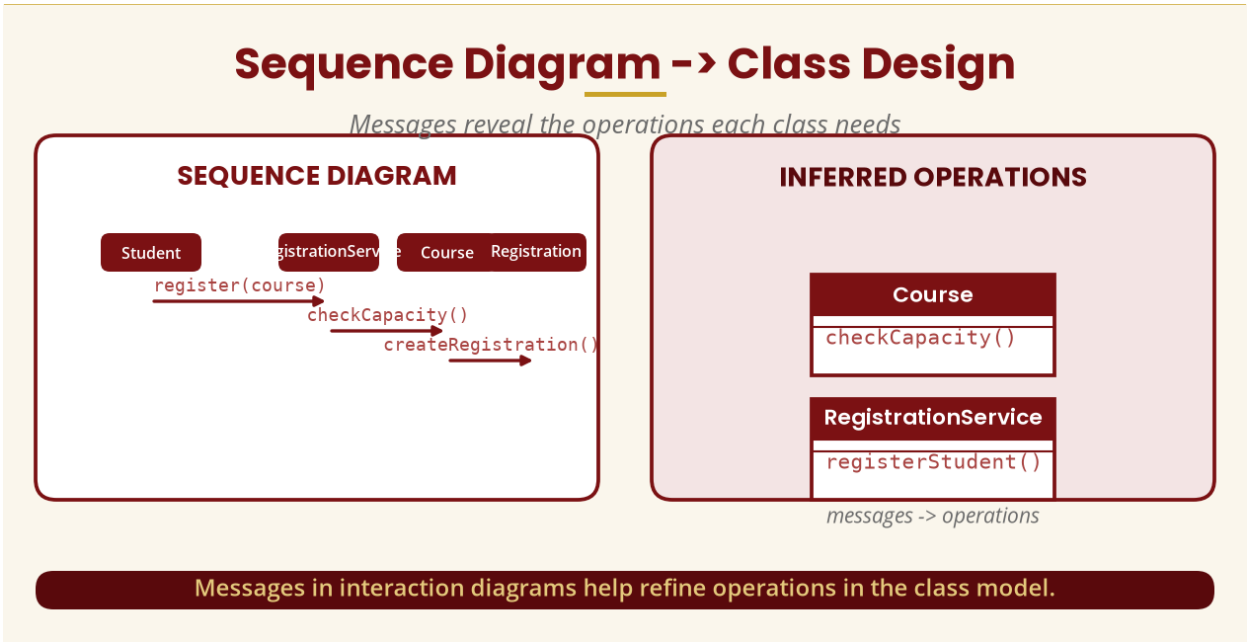

Figure 10 — Sequence diagram -> class design

PART J — MODEL CONSISTENCY

12. Making the Diagrams Agree

Model consistency means the diagrams agree. If the class diagram says `Course.checkCapacity()` but the sequence diagram says `Course.verifyCapacity()`, there is an inconsistency to resolve. Likewise, if the class model says `register()` and the sequence says `enrol()`, the team must decide whether these are the same operation and standardise the terminology. Models should be cross-checked.

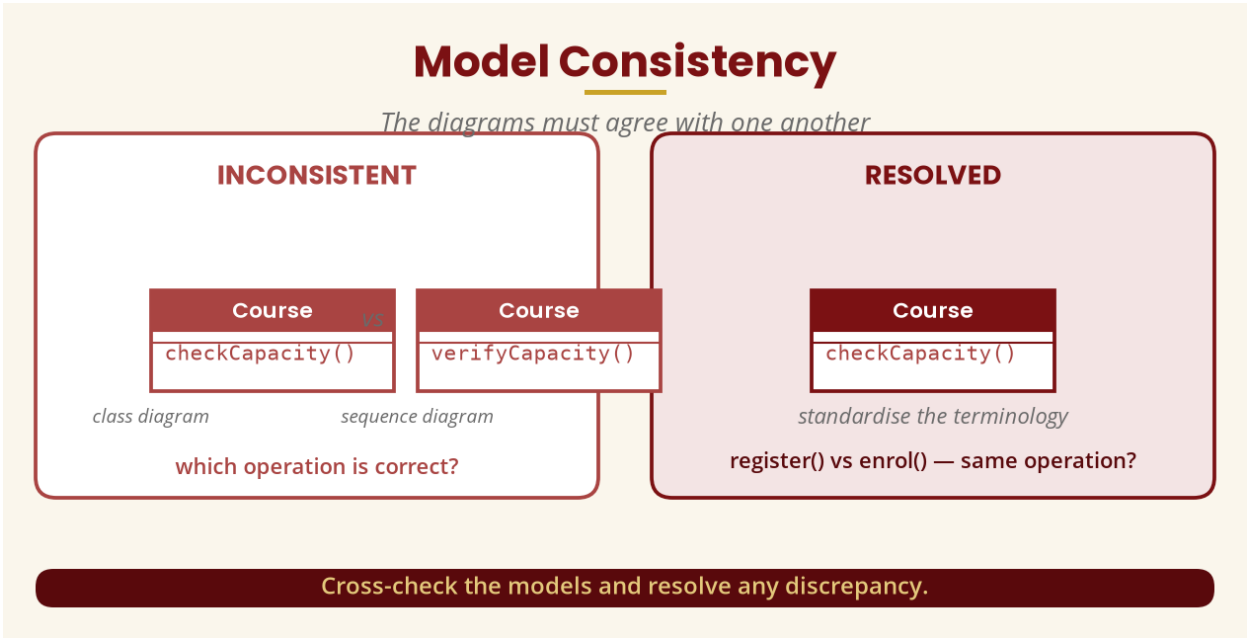

Figure 11 — Model consistency

PART K & L — INTEGRATING THE MODELS

13. Object Model + Dynamic Model

The class diagram shows the structure (Course with capacity, status, registerStudent()); the state machine shows Available -> Full -> Closed. Connecting them: `registerStudent()` may cause Available -> Full when the final seat is taken (e.g. capacity 50, registrations 49 -> 50). **The operation has a relationship with the object's state.**

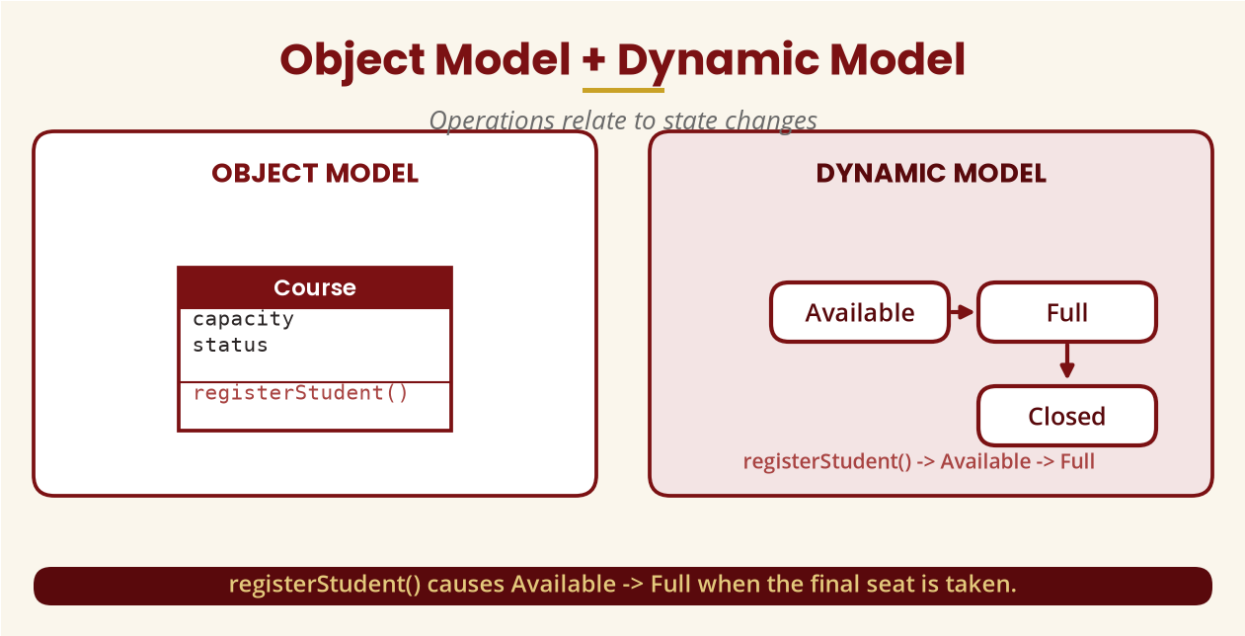

Figure 12 — Object model + dynamic model

14. Object Model + Activity Model

Given the registration workflow, ask *which objects perform these activities?* — Select course -> RegistrationController; Check prerequisite -> Course/RegistrationService; Check capacity -> Course; Create registration -> RegistrationService; Save registration -> RegistrationRepository; Send confirmation -> NotificationService. This transforms a behavioural model into design responsibilities.

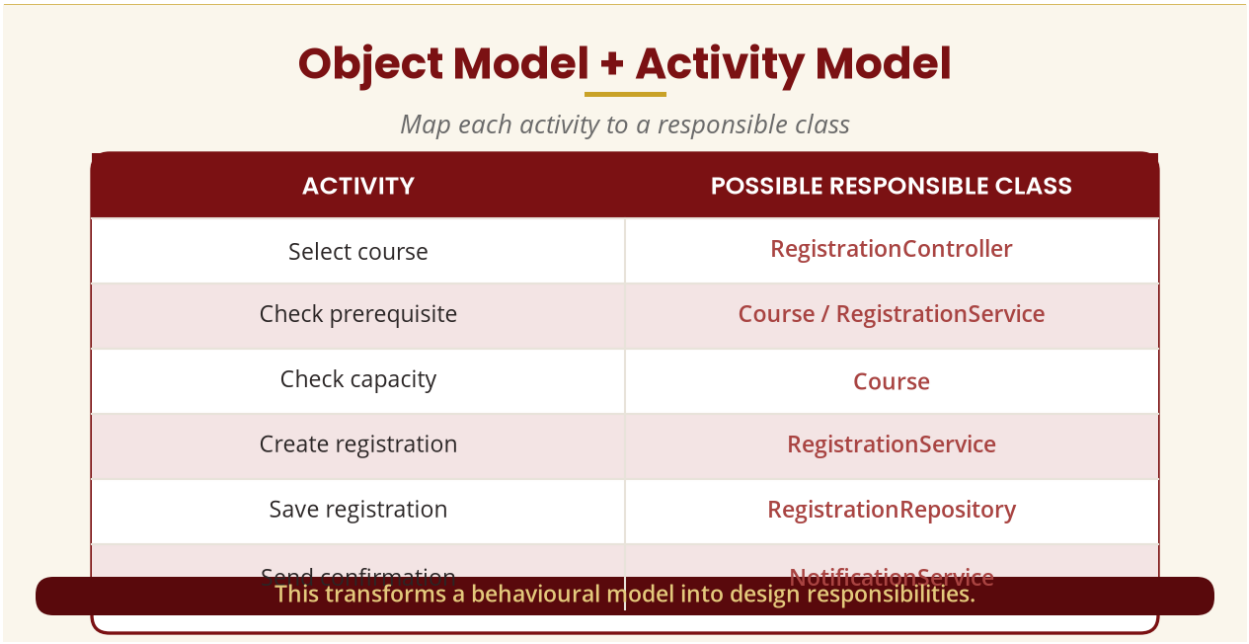

Figure 13 — Activity -> responsibility mapping

PART M & N — USE CASE MODEL AND TRACEABILITY

15. Use Case -> Scenario -> Sequence -> Responsibilities -> Operations -> Design Classes

The use case tells what the user wants; the class model tells what objects are available; the sequence diagram tells how those objects cooperate. This gives one of the most useful workflows in OOAD. **Traceability** follows a requirement through the process: requirement ("students must register for courses") -> use case (Register for Course) -> sequence (Student -> RegistrationService -> Course -> RegistrationRepository) -> classes -> operations (registerCourse(), checkCapacity(), checkPrerequisite(), saveRegistration()) -> code.

PART O–Q — CONTROLLER, SERVICE AND DOMAIN KNOWLEDGE

16. The Controller, the Service and the Domain Object

A **controller** coordinates a system operation initiated by an external actor — RegistrationController receives `registerCourse(studentId, courseId)` and coordinates the process. A controller does **not** mean "do everything" — it must not become a God class checking the database, calculating fees, applying rules, creating HTML and sending SMS; it coordinates RegistrationService -> (Course, Repository) instead.

A **service** provides an application capability — RegistrationService with registerStudent(), dropStudent(), getRegistration() — coordinating check prerequisite -> check capacity -> create Registration -> save -> send confirmation. **Domain objects should contain domain knowledge:** rather than scattering the capacity rule throughout the UI, give Course `isAvailable()` and `hasCapacity()` — assigning responsibilities to the object that has the

information needed to perform them.

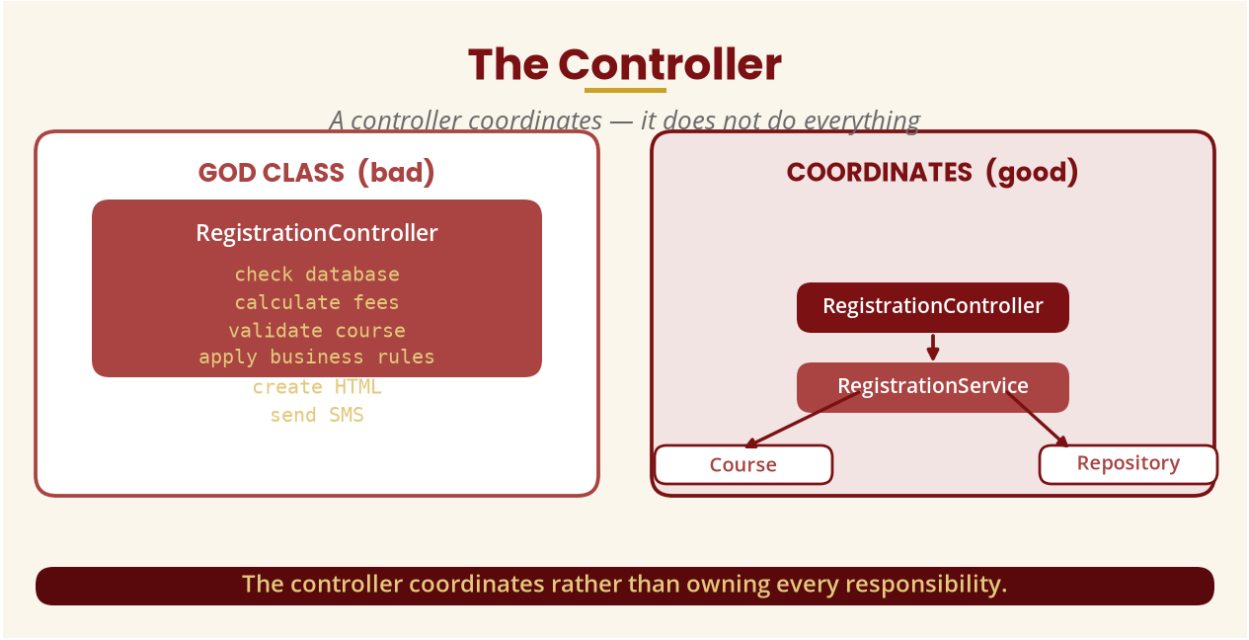

Figure 14 — Controller vs God class

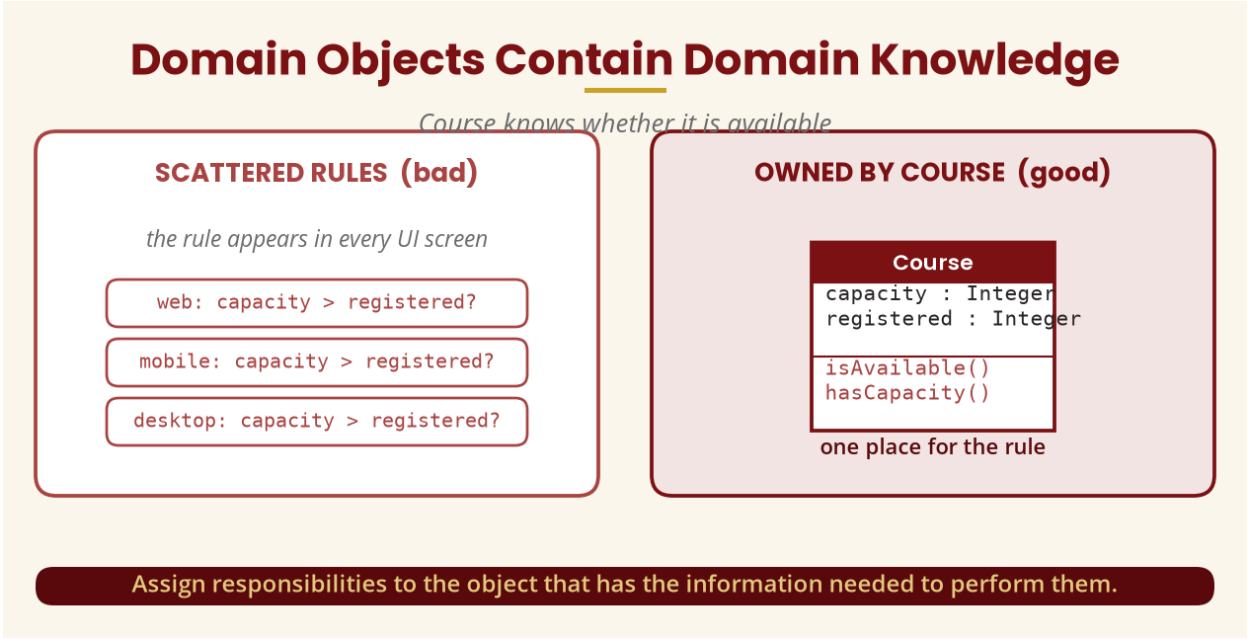

Figure 15 — Domain knowledge

PART R & S — DESIGN REFINEMENT AND DESIGN FOR CHANGE

17. Refinement and Change

Design refinement: the design model is often richer than the analysis model — Student, Course, Registration become Student, RegistrationController, RegistrationService, Course, Registration, CourseRepository, RegistrationRepository and NotificationService. Adding classes is not a sign the analysis was wrong; implementation introduces technical responsibilities (e.g. “save registration” -> RegistrationRepository) that were not obvious during early analysis.

Designing for change: ask *what is likely to change?* — database, payment provider, notification channel, user interface, external API — while the core concepts of Student, Course and registration rules are less likely to change. Put boundaries around the likely changes: instead of RegistrationService -> AirtelMoneyAPI, use RegistrationService -> PaymentGateway -> (Provider A, Provider B).

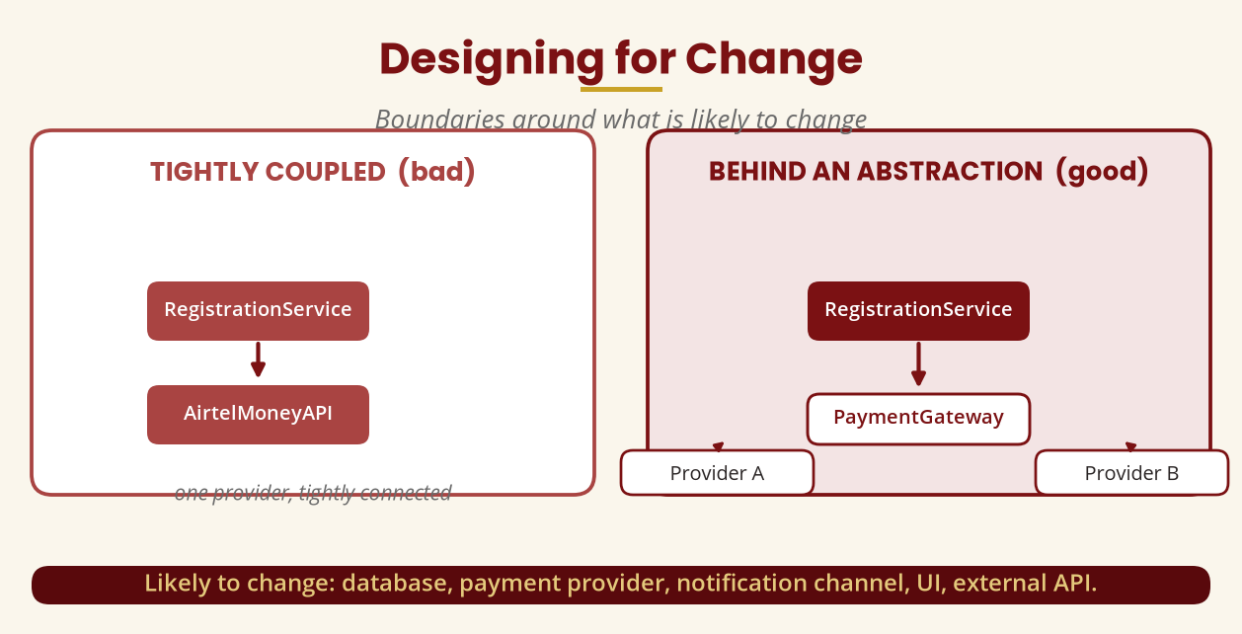

Figure 16 — Designing for change

PART T — DESIGN PATTERNS (outline)

18. The Strategy Pattern

A **design pattern** is a reusable solution structure for a recurring design problem — a proven way of organising a solution, not simply code to copy. The **Strategy pattern**: instead of chaining `if payment == mobile money / else if bank / else if card` everywhere, define a common abstraction `PaymentStrategy` with MobileMoneyPayment, BankPayment and CardPayment — the system works with the abstraction. **Warning:** patterns should be used when they solve an actual recurring problem; otherwise the pattern itself becomes unnecessary complexity.

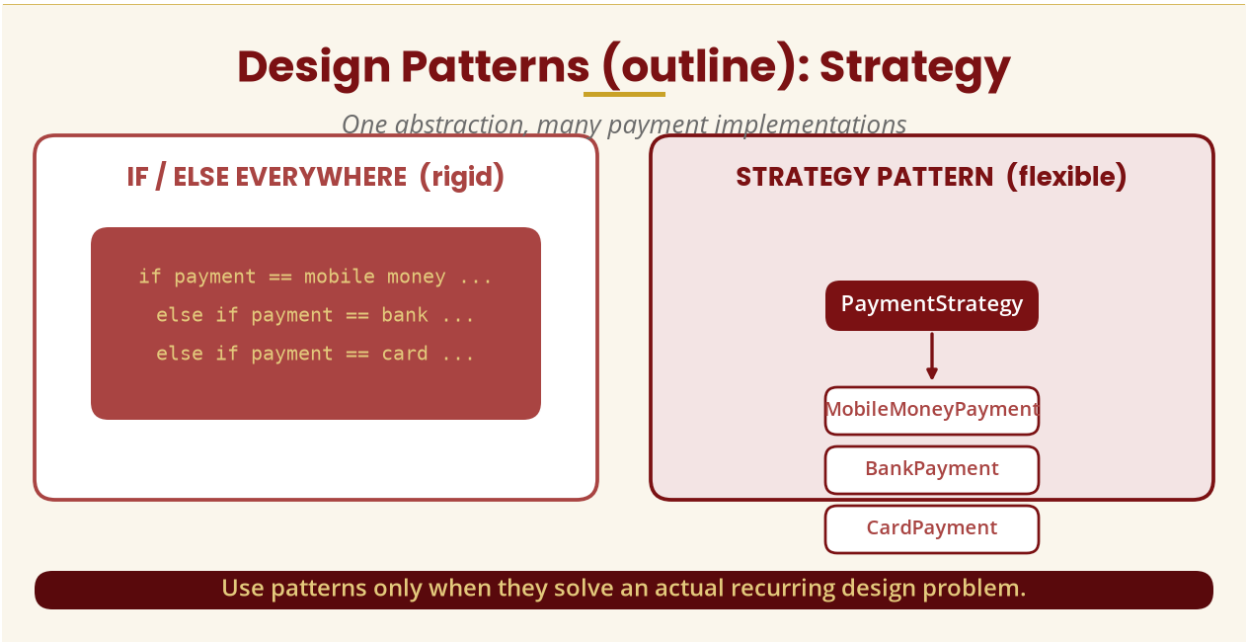

Figure 17 — The Strategy pattern

PART U — INTEGRATED CASE STUDY

19. University Course Registration

Requirement: students must register for available courses. **Use case:** Register for Course. **Analysis classes:** Student — registers -> Registration — for -> Course. **Sequence:** Student -> RegistrationController -> RegistrationService -> checkCapacity() -> Course -> create -> Registration -> save -> RegistrationRepository. **Dynamic model:** Course Available -> Full -> Closed. **Activity:** select course -> check prerequisite -> check capacity -> [available?] -> create -> save -> send confirmation. **Design model:** RegistrationController (+registerCourse()) -> RegistrationService (+registerStudent(), +dropCourse()) -> Course (capacity, status, isAvailable()) and Registration (date, status, confirm()) -> RegistrationRepository (+save(), +find()). **That is what it means to integrate the models.**

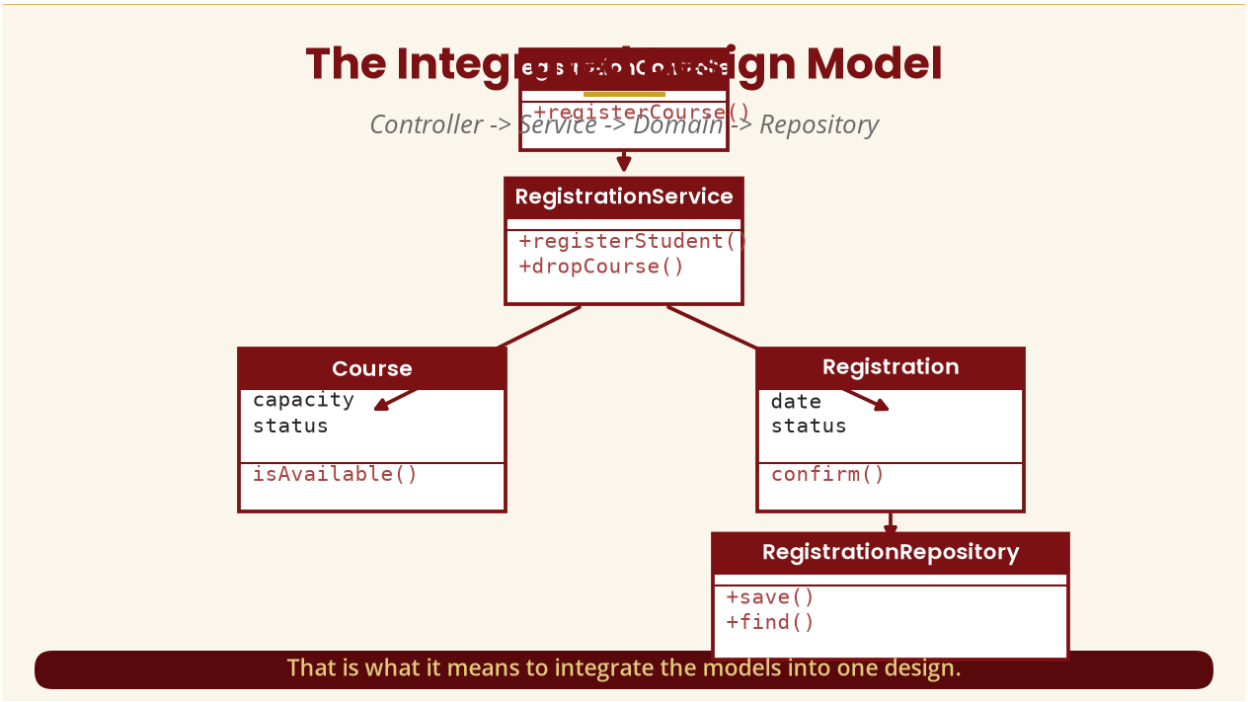

Figure 18 — The integrated design model

PART V — MODEL VALIDATION

20. How Do We Know Our Models Are Correct?

- Does every important use case have supporting classes?
- Does every important sequence message correspond to an operation?
- Do operations belong to sensible classes?
- Do state changes correspond to actual events or operations?
- Do activity flows agree with use-case descriptions?
- Do class relationships support the required interactions?
- Are there duplicated responsibilities, or any class doing too much?

Example of inconsistency: the use case says "students can drop courses", but no class has a drop operation — introduce `RegistrationService.dropCourse()`.

PART W — DESIGN QUALITY CHECKLIST

- **Structure** — classes clearly identified, meaningful relationships, appropriate responsibilities.
- **Behaviour** — sequence diagrams and state transitions make sense; important events represented.
- **Requirements** — every major requirement traces to the design.
- **Architecture** — layers clearly separated; dependencies controlled.
- **Reuse / adaptability / maintainability** — appropriate components reusable; likely changes accommodated; system understandable.

21. Common Student Mistakes

- **Treating UML diagrams as independent drawings** — the sequence diagram should explain how the classes from the class model cooperate.
- **Putting every operation into one class** — a System class doing login(), register(), pay(), sendEmail()... is poor responsibility assignment.
- **Confusing analysis and design** — analysis says Student needs to register; design says the controller receives the request and the service coordinates it.
- **Adding classes without justification** — Manager, Helper, Processor, Utility must have an explained responsibility.
- **Ignoring consistency** — register() vs enrolStudent() must be reconciled.

22. Tutorial and Practical

Tutorial 1 — for "a student selects a course; the system checks prerequisites and capacity; a registration record is created and confirmation sent", identify actors, the use case, analysis classes, design classes, operations, the sequence of interactions and course states. **Tutorial 2** — given a class model (Student, Course, Registration), a sequence message (RegistrationService -> verifyPrerequisite()) and a state model (Course Available -> Full), state what changes make the models consistent. **Tutorial 3** — redesign a Student class doing registerCourse(), sendEmail(), saveToDatabase(), calculateFees(), checkCourseCapacity() and generateReport().

Practical (1 hour): produce an integrated design package for the University Course Registration System — refine the class diagram (attributes, types, operations, visibility, relationships); a sequence diagram (Student, RegistrationController, RegistrationService, Course, Registration, RegistrationRepository); a state model for Course (Created -> Available -> Full -> Closed); an activity diagram; and cross-check at least three consistency relationships (sequence message -> operation, state transition -> event, activity -> responsibility, use case -> design classes).

23. Week 14 Summary and Key Terms

Concept	Simple meaning	Example
Analysis / design	Understand the problem / decide how to implement	Student needs registration / RegistrationService
Object model	What exists	Student, Course
Dynamic model	How things change	Available -> Full
Behavioural model	What the system does / interactions	Register Course
Responsibility	What a class knows/does	Course checks capacity
Attribute / operation	Information held / action performed	courseCode / register()
Controller	Coordinates a request	RegistrationController
Service	Provides an application capability	RegistrationService
Repository	Handles persistence	CourseRepository
Traceability	Follow requirement through design	Requirement -> Code
Model consistency	Diagrams agree	sequence message matches operation

THE MOST IMPORTANT CONCEPT TO TAKE HOME

Requirement -> “Student registers” -> use case “Register for Course” -> scenario -> sequence diagram -> class model -> responsibilities -> design classes -> component/architecture -> implementation. Alongside: **class model <- dynamic model <- behavioural model** — the models must **support one another, not contradict one another**. That is the central lesson of Week 14.

24. Examination-Style Questions

Essay (20 marks). Define object-oriented design and explain how it differs from object-oriented analysis, using a University Course Registration System.

Essay (25 marks). Explain how a sequence diagram can be used to identify and refine operations in a class diagram, with a practical example.

Essay (25 marks). Explain how the object, dynamic and behavioural models complement one another during object-oriented design.

Scenario (30 marks). A university wants an online course registration system: a student selects a course, the system verifies prerequisites and capacity, creates a registration record and sends confirmation. Identify (a) the relevant classes, (b) their responsibilities, (c) major operations, (d) object interactions, (e) relevant course states, and (f) how the UML models remain consistent.

25. References and Further Reading

Primary standards

- Object Management Group (OMG). *Unified Modeling Language (UML), Version 2.5.1.* OMG — the formal UML specification.
- Object Management Group (OMG). *Introduction to OMG Specifications.* Overview of UML diagram categories: class, component, deployment, use-case, sequence, statechart, activity and package diagrams.

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling.*
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML.*
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide.*
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design.*
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements.*
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML.*